CartPilot

Software Engineering Department
Braude College

CartPilot Project Phase A

AI SaaS for Natural-Language Product Discovery in E-commerce Stores

By:
Gal Mitrani
Ofek Amar

Advisor:
Dr. Naomi Unkelos Shpigel

Project Code: 26-2-D-8

GitHub Google Drive

Table of Contents

Common Definitions and Acronyms

Table 1 provides standardized definitions for key technical terms and concepts used throughout this document.

Table 1. Common Definitions and Acronyms

Term	Description
AI	Artificial Intelligence; software techniques that perform tasks requiring reasoning, language understanding, or decision support.
Agent	A software component that receives a goal, chooses tool calls or actions, and returns a result through a controlled workflow.
Agentic	Describes a system in which an LLM-driven component plans and sequences its own actions (which tool to call, which step to take next) rather than only producing text. In CartPilot, this applies narrowly to the backend orchestration step, not to open-ended autonomous behavior.
Reasoning Loop	The bounded sequence of decide → act → observe steps an agent runs before returning a result (e.g., parse → map → retrieve → rank → respond). CartPilot's reasoning loop is deterministic and auditable at each step, in contrast to open-ended agentic planning.
API	Application Programming Interface; a software interface used to exchange structured data between components.
Chrome Extension	A browser-side application that can inject UI and logic into supported web pages through content scripts and extension APIs.
E-commerce Product Discovery	The process by which a shopper finds relevant products through search, filtering, recommendations, or navigation.
Filter Mapping	Translation of natural-language product attributes into the actual filters, URL parameters, or collection paths supported by a specific store.
FR / NFR	Functional Requirement / Non-Functional Requirement.
LLM	Large Language Model; a language model used here for parsing, interpretation, ranking assistance, and response generation.
RAG	Retrieval-Augmented Generation; a generation pattern in which an LLM receives retrieved, relevant context before producing a response.
Schema DB	Database representation of each supported store: platform, collections, filters, filter options, URL patterns, and mapping metadata.
Scraper	A controlled retrieval module that extracts public product information from result pages when API integration is unavailable.
Shopify	The first target platform for the system, because many stores expose predictable collection pages, search URLs, and storefront filters.
SaaS	Software as a Service; a hosted product that can later be provided to merchants or end users through subscription or managed access.

Abstract

Online shoppers routinely struggle to translate a natural product request into the rigid vocabulary of a store's search and filter system, forcing them to guess category names, manually open filters, and repeat searches when keyword matching returns irrelevant results. This friction is especially pronounced on small and medium e-commerce stores, where filter structures, tagging conventions, and search behavior are inconsistent from one store to the next, and where merchants typically lack the resources to build a custom semantic search solution.

CartPilot addresses this problem as an AI-based conversational product discovery system for e-commerce websites, with an initial focus on Shopify stores. Instead of forcing users to discover the exact filter structure of each store, CartPilot allows a shopper to describe a product request in natural language, which the system then translates into site-specific retrieval actions.

The proposed product is delivered primarily as a Chrome extension overlay that appears on the active e-commerce store. The extension sends the user query and the current store context to a backend service. The backend parses the query, loads the relevant site schema, maps query attributes to supported filters, builds a target URL or retrieval request, extracts available product cards, ranks the results, and returns a grounded conversational response. The system avoids relying on the LLM as the source of product truth: product truth comes from the current store inventory and retrieved pages, while the LLM is used for interpretation, reranking support, and explanation. Internally, the backend runs a short, bounded reasoning loop (parse, map, retrieve, rank, respond) rather than a single end-to-end model call, which keeps each step auditable and testable in isolation.

The project combines information retrieval, conversational commerce, web scraping, browser extension engineering, data management, and Retrieval-Augmented Generation. The main engineering contribution is a reusable query-to-filter translation and schema discovery pipeline that can support multiple stores over time. The project will be evaluated using measurable metrics such as query success rate, filter matching accuracy, result relevance, latency, scraping reliability, extension stability, and tracked result click-through behavior.

Keywords: conversational commerce, product discovery, e-commerce search, Shopify, Chrome extension, RAG, LLM, web scraping, information retrieval.

1. Introduction

Online stores usually expose product discovery through a combination of keyword search, category navigation, sorting, and filters. This structure works when the user already knows the exact category names and filter labels used by the merchant. It performs worse when the user has an intent that is natural to describe but difficult to translate into the store interface, such as "comfortable black running shoes under 400", "a lightweight laptop for programming", or "a gift hoodie that is warm but not too expensive". These queries combine category, constraints, style, usage context, and price sensitivity. A typical search box treats most of that language as plain keywords [6,11,15,19].

The practical result is friction. Users must guess the right category, manually open filters, compare product cards, and repeat searches when the store search engine returns irrelevant results. On many smaller stores, search and filters are implemented by a theme, a third-party plugin, or custom code. Therefore, the search behavior is inconsistent from one store to another. This inconsistency is especially visible in Shopify stores, where collections, product tags, variant options, metafields, and theme-specific filter layouts can all influence how products are discovered [14,15,16,19,21].

Existing solutions can be grouped into several categories. The first category is native store search, which is simple and integrated but usually limited to keyword matching, tags, and theme-level filters. The second category is third-party site-search platforms that provide indexing, synonyms, ranking, and analytics, but require merchant integration and may be expensive for small stores. The third category is recommendation engines, which are useful for personalization and related products but do not necessarily solve direct user intent expressed in conversation. The fourth category is generic AI chatbots, which provide a natural interface but are risky unless grounded in store-specific product data [6,10,11,19].

The project focuses on four related problems. First, e-commerce search is often keyword-based and cannot reliably interpret intent, constraints, and synonyms. Second, filters are rigid and require the user to understand how the store models its own catalog. Third, small and medium merchants typically lack the resources to develop advanced semantic search or conversational discovery tools. Fourth, generalized chatbots can hallucinate products or recommend items that do not exist in the current store if they are not grounded in live inventory data. [2,3,4,13,21].

The specific technical problem is therefore: given a natural-language product request and an active e-commerce website, how can the system retrieve and rank products that actually exist in that website inventory, without requiring the merchant to manually integrate a full custom search engine from day one? [15,16,17,20,21].

CartPilot is positioned between these categories, maintaining the intuitive user experience of a conversational assistant while utilizing structured retrieval and site-specific schema mapping to ensure the accuracy of product recommendations [3,14,19,20].

The planned solution is a Chrome extension and backend service. When the shopper visits a supported store, the extension opens an overlay chat. The shopper writes a natural-language query. The backend detects the active domain, loads or discovers the store schema, parses the query into structured attributes, maps those attributes to the store filters, retrieves product result pages, normalizes the extracted product cards, ranks them, and returns a concise answer with product cards and links [2,3,17,20].

CartPilot aims to achieve this by decoupling natural language processing from the retrieval mechanism. Relevance is expected to be improved through deterministic matching and subsequent reranking, though this claim requires empirical validation during the evaluation phase.

The browser extension delivery model allows CartPilot to be utilized without necessitating modifications to the merchant's existing web infrastructure [2,4,13,20,21].

2. Literature review

2.1 Product Discovery and E-commerce Search

E-commerce product discovery is a specific form of information retrieval. Unlike general web search, the searchable objects are products with structured attributes: title, price, brand, variant, color, size, availability, collection, tags, images, and merchant-specific metadata. Product queries are often short but dense. A query such as "black nike shoes under 400" contains category, brand, color, and price constraints. A query such as "laptop for software engineering student" is more semantic and requires interpretation of use case, expected specifications, and budget [6,11,15,16,21].

Information retrieval literature distinguishes between query understanding, candidate retrieval, ranking, and evaluation. In e-commerce, ranking quality is not only a relevance problem; it is also connected to business metrics such as clicks, add-to-cart behavior, purchase rate, availability, and revenue. Applying learning-to-rank methods in e-commerce is particularly difficult because product features, user feedback signals, and relevance judgments are noisy and business-dependent. This supports the need for a controlled, measurable ranking layer rather than a pure LLM answer [11,21].

Product search relevance modeling also shows that classical retrieval features and product metadata are still important, since comparisons of traditional retrieval models with embedding-based approaches for query-product relevance support preserving structured product fields rather than collapsing all product retrieval into free-text generation [16,20].

2.2 Existing E-commerce Discovery Tools

Several types of discovery tools exist in e-commerce, each with distinct capabilities and limitations. Native store search is tightly integrated but keyword-dependent; filter navigation is precise but requires users to know the store structure; third-party search platforms add advanced features but require merchant setup; recommendation widgets support personalization but lack explicit intent handling; and generic AI chatbots offer flexible conversation but risk hallucinating products. Table 2 compares five existing product-discovery approaches - native store search, filter navigation, third-party search apps, recommendation widgets, and generic AI chatbots - against their typical strengths, weaknesses, and how CartPilot relates to or improves on each.

Table 2. Comparison of Existing E-commerce Discovery Tools

Tool Type	Typical Strength	Typical Weakness	Relation to the system
Native store search [6,19]	Already integrated with the store and product catalog.	Often keyword-based and inconsistent across themes.	The system uses it as one retrieval path but adds language understanding.
Filter navigation [14,19]	Precise when the user knows the filter structure.	Requires manual interaction and exact category/filter knowledge.	The system maps natural language into filters automatically.
Third-party search apps [17,20]	Can provide indexing, synonyms, analytics, and ranking.	Requires merchant installation and commercial integration.	The system starts as an overlay and can later become SaaS integration.
Recommendation widgets [10,11]	Useful for related products and personalization.	Usually less direct for explicit search intent.	The system focuses on explicit user intent first.
Generic AI chatbot [2,19]	Natural conversation and flexible wording.	May hallucinate products without grounding.	The system grounds the response in retrieved store products.

The comparison in Table 2 justifies the design of CartPilot as an agentic, schema-grounded assistant rather than a generic chatbot. The weaknesses identified above - the absence of automated inventory synchronization, the lack of intelligent product discovery tools, and the tendency of generic AI chatbots to hallucinate product suggestions that do not reflect actual store

inventory motivate an architecture where the agent's actions (schema lookup, filter mapping, retrieval) are explicit and checkable, not just its final answer.

The assistant must be connected to the current store schema and retrieval process. Otherwise, it may provide fluent but incorrect product suggestions. Conversely, implementing only a scraper without conversational interpretation would not solve the user experience problem. [3,10,13,14,19].

2.3 Conversational Commerce and Conversational Recommender Systems

Conversational recommender systems are relevant to CartPilot's design because they model user preferences through multi-turn interaction, directly addressing the intent-elicitation problem identified in Section 1, that a single query rarely captures the full set of constraints a shopper has in mind. Research in this area consistently identifies challenges such as preference elicitation, dialogue understanding, exploration versus exploitation, and evaluation. It is also well established that conversational recommendation is not only about final recommendations but also about how the system asks clarifying questions, represents knowledge, and evaluates success. When a query is ambiguous, an effective conversational system should either return the most relevant grounded results or prompt the user with a targeted clarifying question in order to narrow down the search intent [10,11,19,20].

2.4 Query Understanding and Natural Language Interpretation

Query understanding is the bridge between natural user language and structured retrieval. E-commerce queries often contain product attributes and shopping intent embedded in ordinary language, which directly matches the system requirements. Language interpretation is facilitated by a hybrid query parser - a component that converts a free-text shopper query into a structured attribute object (category, brand, color, size, price range, and intent modifiers such as cheap or premium), combined with Large Language Models (LLMs) for handling ambiguous or synonym-rich phrasing [10,17,20].

The parser can be implemented as a hybrid component. Rule-based parsing handles common patterns and provides deterministic behavior. LLM parsing handles messy wording, synonyms, and use-case phrasing. The parser output should be a typed object, not free text.

2.5 Retrieval-Augmented Generation and Grounded Responses

Retrieval-Augmented Generation (RAG) integrates language models with external, retrieved context, combining parametric model knowledge with externally retrieved documents to reduce reliance on the model's own memory [13]. Recent RAG surveys distinguish standard RAG from agentic RAG, in which an autonomous agent plans retrieval steps, decides when to call which tool, and reasons over multiple retrieval rounds before answering [13]. CartPilot adopts the same underlying idea — treating retrieval as a sequence of decisions rather than a single lookup — but keeps the reasoning loop narrow and rule-governed (fixed steps: parse, map, retrieve, rank, respond) rather than letting the model choose its own tool sequence. This trade-off favors auditability and grounding over open-ended flexibility, which matches NFR-04 (Grounding) and NFR-06 (Maintainability).

The RAG design should be narrow and controlled. Product cards returned to the user must come from retrieval and normalization, not from model memory. The LLM can summarize why products match the query, explain trade-offs, and ask follow-up questions, but it must not invent unavailable products, unsupported discounts, or hidden inventory. This is a key non-functional requirement for reliability and trust [3,4,13,17,20].

A useful response format is therefore hybrid: a short natural-language answer followed by structured product cards. Each card includes title, price, image, store URL, matched attributes, and a tracked redirect link. This format supports both user experience and evaluation, because the system can log which retrieved product was shown and clicked [3,14,17,19,20].

3. Methodology

3.1 Stakeholder Analysis

To ground the system's requirements in real merchant and shopper needs, we conducted five semi-structured interviews with independent e-commerce store owners, selected to represent a deliberately diverse range of catalog sizes, product domains, and technical sophistication. Table 3 summarizes the five participants.

Table 3. Interview Participants

ID	Domain	Technical background	Years of experience	Platform
1	Computer hardware	Technical	20+	Website
2	Streetwear & exclusive fashion	Non-technical	3+	Shopify + Instagram
3	Biomedical/clinical equipment	Technical	10+	Shopify
4	Footwear & fashion	Light-technical	2+	Website + Instagram
5	Candles & essential oils	Non-technical	5+	Shopify

This mix of technical and non-technical merchants across five distinct product domains was chosen intentionally: it surfaced pain points shared across e-commerce in general unstructured natural-language search, reliance on live inventory grounding, response speed, and low-effort integration as well as domain-specific constraints, such as regulatory caution around biomedical equipment, style-based rather than filter-based search in fashion, and price/stock volatility in hardware. These findings were translated into design priorities for the system. Table 4 summarizes the five identified stakeholder groups and how the proposed system addresses each group's needs.

Table 4. Stakeholder Analysis

Stakeholder	How the system Helps
Online shoppers [6,11,14,19]	Find relevant products faster, without learning the store filter structure.
Small and medium merchants [11,15]	Improve discovery and engagement without a full custom search project.
Affiliate/operator side [6,11]	Track query-to-click behavior and validate business value.
Development team [4,7,13]	Build a reusable multi-site retrieval system and demonstrate software engineering competence.
Academic reviewers [4,7]	Evaluate the project through measurable engineering outputs, literature grounding, and validation metrics.

These characterizations draw on the interview findings (Section 4.4) and the discovery-tool limitations identified in the literature review (Section 2.2). The primary user stakeholder, the online shopper, wants fast product discovery with low cognitive effort indifferent to how filters or URLs are structured, but sensitive to relevance, availability, and ease of comparison. A successful system should minimize manual browsing steps and present results the user can act on immediately [6,11,14,19]. The future merchant stakeholder wants higher product exposure and insight into user search behavior; as a future SaaS product, this value comes largely from

analytics on unsupported queries, missing categories, popular attributes, click-through and conversion signals that help merchants refine their catalog and filters [11,15].

The project team stakeholder needs a system feasible within the academic scope: starting with one or several Shopify stores, expanding toward multi-site support, and demonstrating a full end-to-end flow rather than a broad but shallow platform [4,7,13].

3.2 Research Questions

The following research questions were derived from the technical problem defined in Section 1 and the discovery-tool limitations identified in the literature review (Section 2.2), refined through the stakeholder interviews.

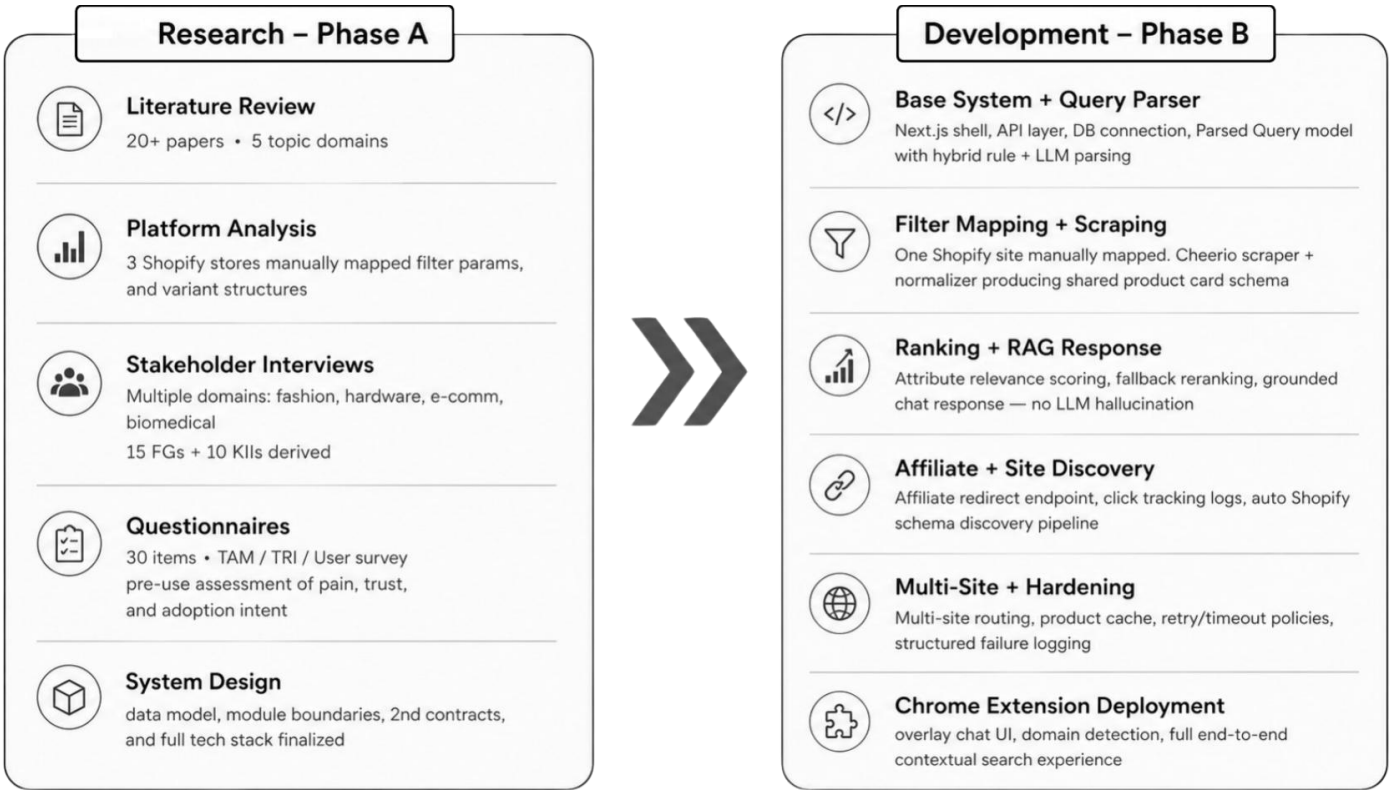
Can natural-language product requests be translated into structured product search attributes with sufficient accuracy for an e-commerce setting? [10,16,19,21]

Can dynamic retrieval from a store produce accurate product cards without requiring a full product pre-index for every supported merchant? [2,9,17]

Can a Chrome extension overlay improve the product discovery flow without disrupting the original store interface? [17,18]

3.3 Research and Development Method

The project will use an iterative engineering process. First, we reviewed literature, existing tools, and conducted interviews with five e-commerce store owners across diverse domains (Section 3.1, Table 3). Second, we defined the minimal supported product attributes and data schema. Third, one Shopify store will be manually analyzed to understand collections, filters, and URL behavior. Fourth, the system will implement a single-site end-to-end flow. Fifth, the flow will be generalized into reusable modules and validated on additional sites [2,4,7,13].



The research method follows a mixed-methods design, combining quantitative engineering evaluation (benchmark queries, automated tests, latency and accuracy metrics) with qualitative evaluation (user questionnaires and interviews), consistent with established practice for evaluating conversational and recommendation systems [10,19]. The main evidence will come from prototype behavior, benchmark queries, automated tests, and user questionnaires. The questionnaires measure perceived usefulness, ease of use, relevance, trust, and preference over manual browsing. These questionnaires are placed in the appendix after the main book [4,11,16]. Stakeholder feedback is collected through demonstrations, short user tasks, questionnaire responses, and review of selected queries — for example, comparing time and steps to complete the task 'find a black hoodie under 300' manually versus through the system [4,11].

3.4 Data Collection During Development

Table 5 details the data sources driving development, from manual store inspection to user questionnaires.

Table 5. Data Collection During Development

Data Source	Purpose	Expected Output
Manual store inspection	Understand Shopify filters, collection URLs, and product card structure.	Initial store schema and mapping rules.
Saved HTML fixtures	Test scraping and extraction without repeatedly hitting live sites.	Stable integration tests.
Benchmark query set	Measure parsing, mapping, and ranking quality.	Accuracy metrics and regression suite.
Click logs	Measure whether returned products receive user action.	CTR and query-to-click linkage.
Questionnaires	Measure user perception and qualitative issues.	Appendix questionnaire results and discussion.

4. Project Requirements

4.1 Storefront Filtering and Site Schema Discovery

E-commerce platforms expose filtering mechanisms — collection pages, search endpoints, product tags, variant options, price filters, and vendor fields — but these vary even between stores on the same platform, differing in filter labels, URL structures, and tagging conventions. The initial implementation targets Shopify for its well-documented, predictable filter structures, with an architecture designed to generalize to other platforms exposing similar capabilities.

To handle this variability, CartPilot maintains a per-site schema — domain, platform, collections, filter groups, URL patterns, and synonyms — rather than assuming a universal filter model. Schema discovery reads publicly available store pages at a controlled request rate, normalizes the extracted data, and persists results with caching and fallback behavior, within the legal and ethical boundaries discussed in Section 9.2 [2,5,8,9].

4.2 Browser Extensions as Delivery Mechanism

A Chrome extension overlays a chat panel on the existing store without requiring merchant-side changes: it detects the active domain, injects the UI, collects the query, and communicates with the backend. Built on Manifest V3, the current Chrome extension standard, CartPilot follows a minimal-permission architecture — operating only on supported domains, communicating only with the designated backend, and avoiding collection of unrelated browsing data. Because the extension executes in the user's browser, it must avoid storing sensitive data client-side, use HTTPS for all backend communication, and validate backend responses before rendering them.

4.3 Technology Foundations

The implementation stack is Next.js (App Router), TypeScript, shadcn/ui, Zod, and Firebase (or another managed database), with a backend retrieval layer and optional LLM integration [22,23,24,25,26]. Firebase Firestore's document model fits store schemas, query logs, and click events [23]; Zod provides runtime validation for parsed queries and API payloads [25]. The core engineering challenge is not tool selection but defining module boundaries — UI, parsing, schema management, scraping, ranking, and analytics — so that each can be tested independently, with room to migrate from Firebase to PostgreSQL later if relational analytics needs grow.

4.4 Requirement Sources

Requirements were derived from the five stakeholder interviews described in Section 3.1 (Table 3), covering intent-to-filter mapping, stock availability, response speed, extension usage, analytics, and recommendation reliability. These findings were translated into the functional and non-functional requirements defined in the following sections.

4.5 Functional Requirements

Table 6 enumerates the fifteen functional requirements that define the system's core capabilities.

Table 6. Functional Requirements

ID	Requirement	Type
FR-01	The system shall allow a user to submit a natural-language product query from a chat interface.	Core UX
FR-02	The system shall detect or receive the active e-commerce domain from the Chrome extension.	Context detection
FR-03	The system shall parse user queries into structured fields such as category, brand, color, size, style, price range, and free-text intent.	Query parsing
FR-04	The system shall validate parsed query objects before retrieval.	Data integrity
FR-05	The system shall store and load site-specific schemas containing platform type, filters, options, collections, and URL rules.	Schema management
FR-06	The system shall map parsed query fields to the active site schema.	Filter mapping
FR-07	The system shall build a target search or collection URL according to the mapped filters.	URL builder
FR-08	The system shall retrieve product results from the active store using controlled scraping, HTTP retrieval, or platform-supported paths.	Retrieval
FR-09	The system shall normalize product results into a common product card format.	Normalization
FR-10	The system shall rank product cards according to relevance to the parsed query.	Ranking
FR-11	The system shall generate a grounded conversational response based only on retrieved products and known schema data.	RAG / response
FR-12	The system shall provide fallback behavior for unsupported filters, empty results, parser failure, and retrieval failure.	Error handling
FR-13	The system shall track result clicks through a redirect endpoint for analytics and affiliate readiness.	Analytics
FR-14	The system shall support onboarding of at least one unknown Shopify store through schema discovery or manual configuration.	Site discovery
FR-15	The system shall expose developer/admin visibility into logs, failures, supported stores, and schema status.	Operations

4.6 Non-Functional Requirements

Table 7 presents the ten non-functional requirements governing system performance, security, and usability.

Table 7. Non-Functional Requirements

ID	Category	Requirement
NFR-01	Accuracy	At least 80% of benchmark queries should extract the main category and at least one relevant attribute correctly in the first prototype benchmark.
NFR-02	Latency	Median query-to-results latency should be below 5 seconds for cached or simple retrieval and below 10 seconds for uncached live retrieval.
NFR-03	Reliability	The extension UI and backend should handle parser, network, and scraper failures without crashing.
NFR-04	Grounding	The response must not include products that were not retrieved from the active store or validated cached data.
NFR-05	Scalability	The architecture should support multiple store schemas without cross-store filter leakage.
NFR-06	Maintainability	Parser, mapper, URL builder, scraper, normalizer, ranker, and response generator should be separate modules with tests.
NFR-07	Security	The extension should request minimal permissions and communicate with the backend using HTTPS.
NFR-08	Privacy	The system should avoid storing unrelated browsing data and should log only data needed for product discovery metrics.
NFR-09	Ethical retrieval	The scraper should use rate limits, caching, and fallback logic to reduce unnecessary load on target websites.
NFR-10	Usability	The overlay should be simple, mobile-aware where possible, and should not block normal site navigation.

4.7 Interface Requirements

Table 8 defines the interface contracts between the system's main components, covering inputs, outputs, and technical notes.

Table 8. Interface Requirements

Interface	Input	Output	Notes
Extension to API	Domain, user query, session ID, optional locale.	Request ID and response payload.	Main runtime request path.
API to Parser	Raw query text.	ParsedQuery object.	Validated with Zod.
Mapper to Schema DB	Domain and parsed fields.	Filter mapping candidates.	Must isolate site schemas.
URL Builder to Worker	Mapped filters and URL rules.	Target retrieval URL.	Supports fallback and relaxation.
Worker to Store	HTTP/browser request.	HTML or structured response.	Controlled retrieval only.
Normalizer to Ranker	Raw product cards.	Normalized product cards.	Shared schema for response and tests.
Redirect Endpoint	Tracked result ID.	Redirect to the merchant product page.	Enables affiliate and click analytics.

5. Engineering Process and AI-Assisted Development

5.1 Development Phases

Table 9 lays out the eleven incremental development phases, from initial research to full Chrome extension delivery.

Table 9. Development Phases

Phase	Name	Main Work	Deliverable
0	Research	Finalize architecture, analyze Shopify filters, define DB schema.	Architecture draft, initial schema, literature notes.
1	Base System	Build Next.js frontend, API skeleton, DB connection.	Working shell and backend health route.
2	Query Parsing	Implement ParsedQuery using rule-based logic and optional LLM parsing.	Parser tests and benchmarks.
3	Filter Mapping	Map one target Shopify site manually and generate URLs.	First query-to-filter flow.
4	Scraping	Extract product data and normalize results.	Product cards from live or saved HTML.
5	Ranking + Response	Rank products and generate grounded chat responses.	Usable end-to-end search output.
6	Affiliate Layer	Add redirect endpoint and click tracking.	Query-to-click analytics baseline.
7	Site Discovery	Detect Shopify and extract filters.	Schema persistence for unknown stores.
8	Multi-Site	Support multiple domains and manual/automatic onboarding.	Multi-site routing foundation.
9	Hardening	Add retries, cache, logs, timeouts.	Stable retrieval flow.
10	Chrome Extension	Build overlay UI, domain detection, backend connection.	Contextual on-site assistant.

5.2 AI Tools Used in Development

AI tools will be used as development accelerators, not as unverified authorities. The project will use AI for literature exploration, draft writing, requirement extraction, architecture review, code scaffolding, test-case generation, prompt refinement, and questionnaire refinement. Every AI-generated technical output should be reviewed by the team and validated through tests or source checking.

5.3 Expected Development Products

Working Next.js application shell with chat UI and result card layout (POC).
Backend API layer with route handlers for query, stores, schema, retrieval, redirect, and health checks.
Database schema for stores, filters, filter options, query logs, product cache, and click events.
ParsedQuery model and parser tests covering normal, ambiguous, invalid, and mixed-language inputs.
One manually mapped Shopify store and at least one additional store onboarding scenario (MVP).
Scraper/normalizer with saved HTML fixtures and integration tests.
Ranking module with benchmark query set and measurable relevance criteria.
Chrome extension shell with overlay chat UI and domain detection.
Testing report showing unit, integration, E2E, and performance smoke results.

5.4 Expected Challenges

The largest technical challenge is inconsistent store structure. Even within Shopify, themes differ in markup, filter layout, and URL behavior. The project mitigates this by separating schema discovery from query processing and by persisting a per-site schema. The second challenge is hallucination risk. This is mitigated by allowing the LLM to explain retrieved results but not invent

products. The third challenge is latency. Live scraping can be slow, so the system needs caching, request timeouts, and partial fallback responses.

A fourth challenge is legal and ethical retrieval. The project should use public pages, avoid authentication bypass, apply rate limits, cache repeated requests, and provide a path for future merchant-approved integration. The fifth challenge is scope control. The team should avoid building a complete universal shopping assistant in Phase A. The target should be a demonstrable, testable product discovery pipeline [2,5,8,9].

5.5 Development Standards

Use TypeScript types and Zod schemas for API boundaries.
Keep scraping logic isolated from UI code.
Use fixtures for repeatable tests and avoid relying only on live websites during testing.
Log module-level failures with domain, query ID, error category, and timing.
Prefer deterministic fallback behavior over silent failure.
Maintain a small benchmark query set that grows during development.

6. Solution Architecture and Operation

6.1 High-Level Architecture

The architecture is built around a controlled pipeline. The client layer contains the Chrome extension and optional web demo/admin interface. The API layer validates requests and routes them to the agent workflow - a bounded reasoning loop rather than an open-ended autonomous agent. The agent workflow performs parsing, schema mapping, URL construction, ranking, and response generation, in that fixed order. A background worker handles discovery and retrieval tasks that may be slower or less deterministic. The data layer stores schemas, cached product results, vector/knowledge context, logs, and click analytics.

This architecture intentionally separates LLM-dependent steps from product retrieval. The LLM can help understand and explain, but the actual product results are produced by schema-aware retrieval and normalization. This separation makes the system testable and reduces the chance of hallucinated inventory.

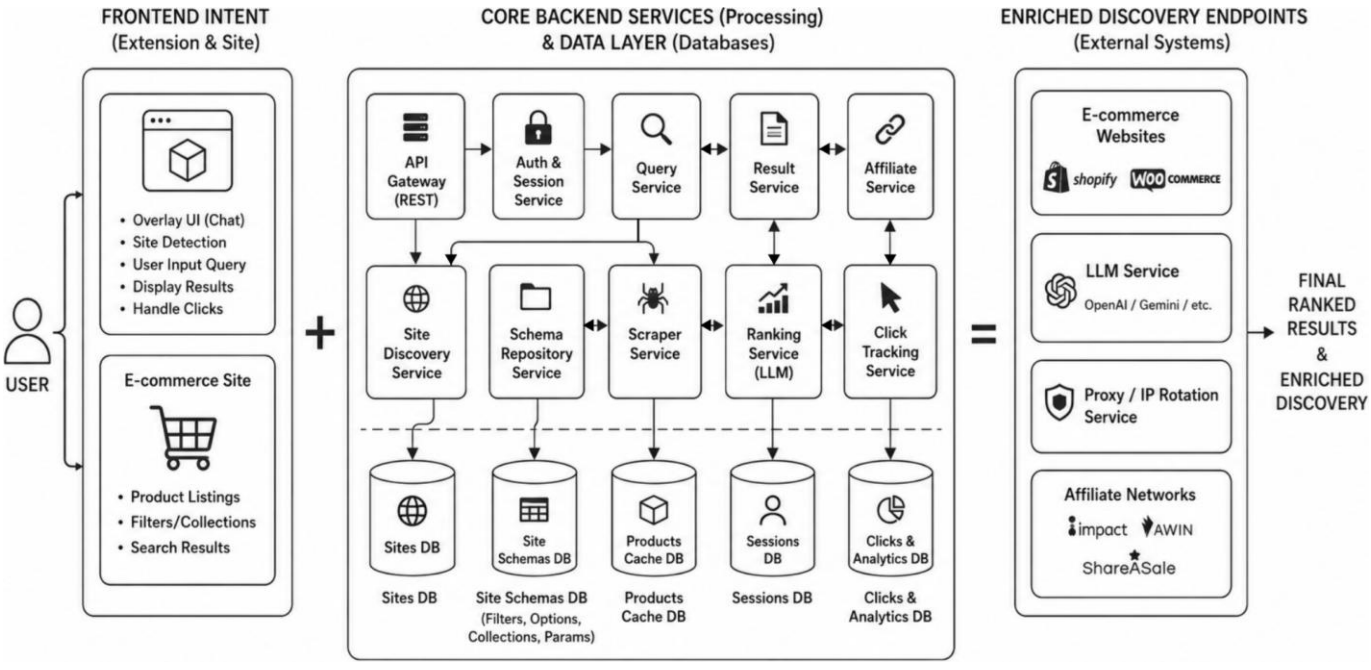


Figure 1. High-level System architecture

6.2 Use Case

A shopper interacts with CartPilot by entering a natural-language query, such as “waterproof winter jacket under $200.” The system parses these constraints and maps them to the store's specific filter parameters (e.g., collection/jackets?filter.price.max=200). Relevant products are retrieved, normalized, and ranked, presenting a curated list in the chat overlay. This automates the discovery process, bypassing manual navigation of store menus and filter hierarchies.

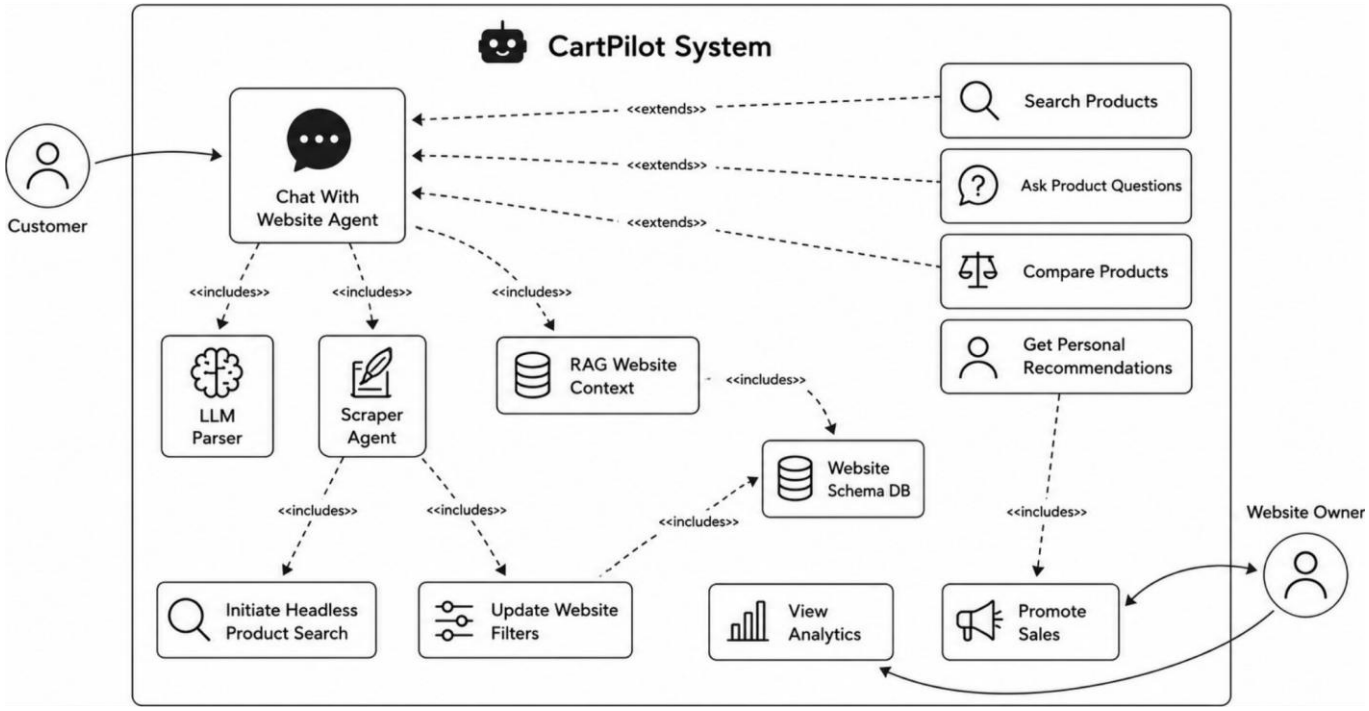


Figure 2. System use case diagram

6.3 Activity Flow

The activity diagram begins with the user query submitted via the extension. The agent identifies the domain and retrieves the site schema. The query is parsed into attributes, mapped to store-specific filters, and used to build a retrieval URL. Products are then extracted, normalized, ranked, and presented. A fallback loop handles insufficient results by relaxing constraints, trying broader collections, or requesting clarification, ensuring robustness against inconsistent store structures.

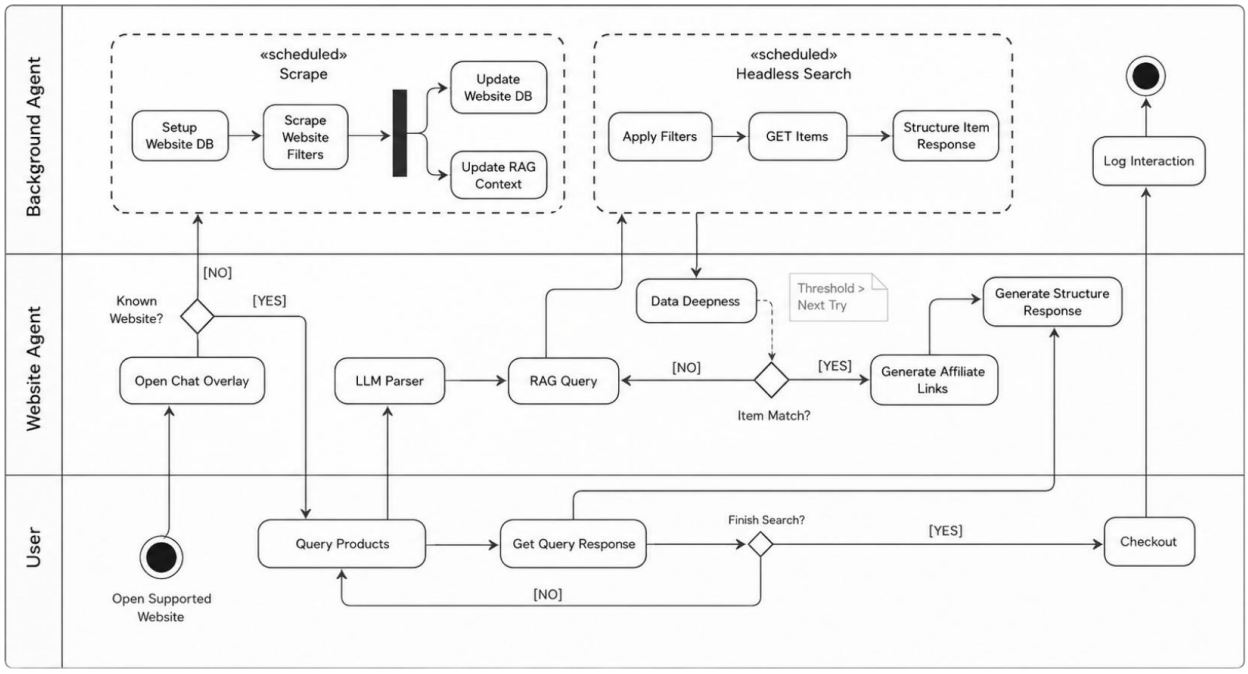


Figure 3. Activity diagram for known and unknown sites

6.4 Module Descriptions

Tables 10 and 11 map each system module to its core responsibility, expected input/output, anticipated failure scenario, and mitigation approach. The project's runtime agent is itself AI-assisted: it receives a user query and store context, then coordinates parsing, mapping, retrieval, ranking, and response generation. Internally, several of these modules are implemented as LLM-driven agents with strict, bounded responsibilities rather than as a single open-ended autonomous agent (Figure 4).

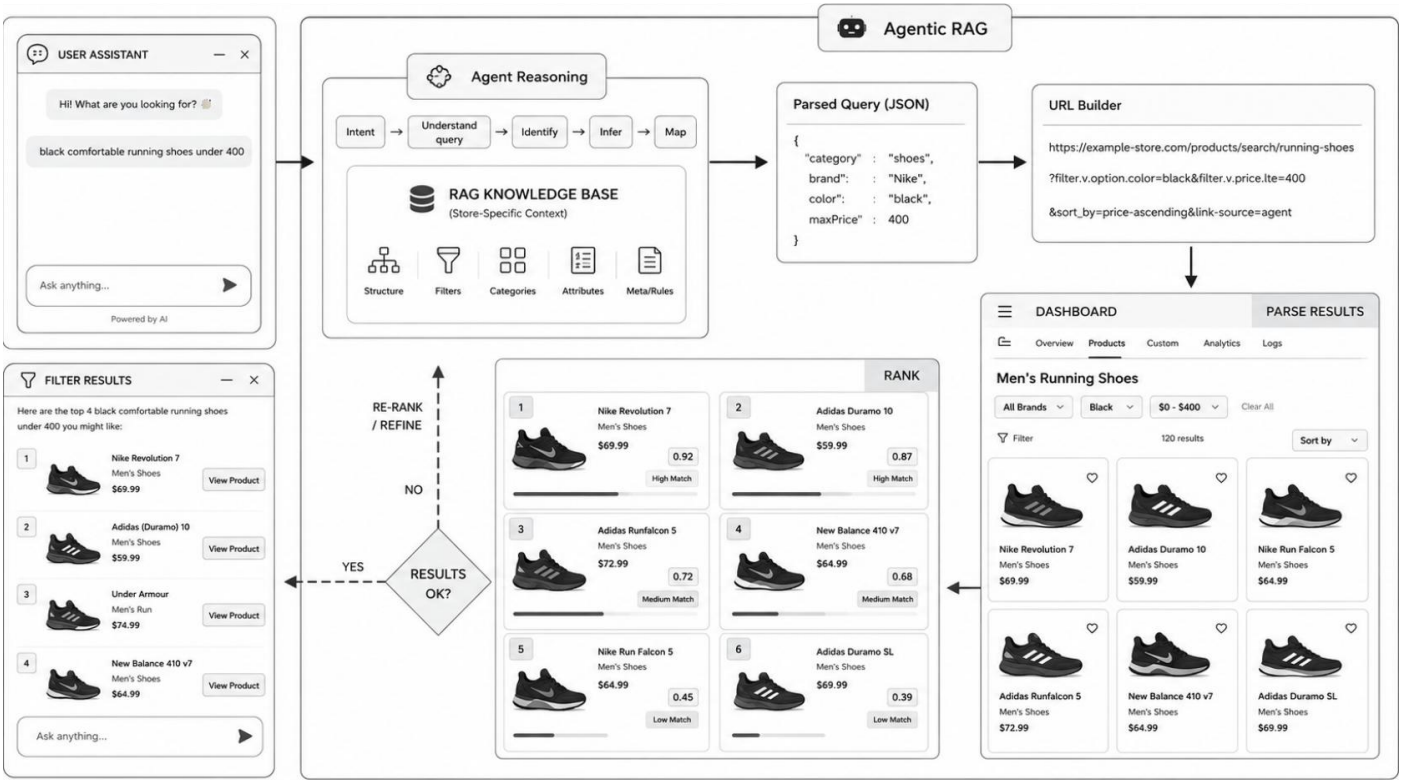


Figure 4. AI agent and service pipeline

Table 10. Interface & Support Modules

Module	Responsibility	Output	Failure Mode and Mitigation
Extension UI	Collect query and show results on top of the store.	Rendered result panel with product cards.	Main failure: the overlay conflicts with site layout. Mitigation: scoped CSS, a toggle button, and a non-blocking panel.
API Gateway	Validate request and coordinate services.	Validated request routed to the agent workflow.	Main failure: malformed request or abuse. Mitigation: Zod validation, rate limits, and error responses.
URL Builder	Create a valid search or collection URL from the mapping plan.	Target retrieval URL.	Main failure: bad parameter or unsupported combination. Mitigation: unit tests per platform and stored rules.
Normalizer	Convert raw cards to a shared structure.	Normalized product card.	Main failure: missing image, price, or availability data. Mitigation: nullable fields and validation.
Redirect/Analytics	Track clicks and redirect to merchant.	Redirect and logged analytics event.	Main failure: broken product URL. Mitigation: URL validation and a safe error page.

Table 11. Core Agent Pipeline

Module	Responsibility	Output	Failure Mode and Mitigation
Query Parser	Extract product intent and attributes from natural language.	Validated ParsedQuery object.	Main failure: ambiguous or unsupported wording. Mitigation: hybrid rules plus LLM fallback with a confidence score.
Schema Repository	Detect platform, identify filters and collections, and store per-site rules.	Site schema draft.	Main failure: missing or stale schema. Mitigation: discovery refresh and manual override.
Filter Mapper	Map ParsedQuery attributes to supported site filters.	Mapping plan and confidence score.	Main failure: no matching filter option Mitigation: synonyms, fuzzy matching, and graceful ignore.
Retriever	Fetch product result pages safely.	Raw product data.	Main failure: timeout or blocked request. Mitigation: timeouts, caching, retries, and saved fixtures.
Ranker	Score exact, partial, and fallback matches.	Ranked product list.	Main failure: poor ranking when filters are missing. Mitigation: attribute scoring and reranking.
Response Generator	Generate a concise, grounded answer.	Chat response and product cards.	Main failure: hallucinated text. Mitigation: only use retrieved product context.

6.5 Data Model Overview

Table 12 describes the core database entities underlying the system, including their key fields and functional roles.

Table 12. Data Model Entities Overview

Entity	Key Fields	Purpose
Store	id, domain, platform, status, lastDiscoveredAt	Identifies a supported or discovered store.
FilterGroup	storeId, label, canonicalName, paramName, type	Represents a filter category such as color, size, vendor, price.
FilterOption	groupId, label, canonicalValue, urlValue, synonyms	Represents possible filter values.
ParsedQueryLog	queryId, rawText, parsedFields, confidence, timestamp	Supports debugging and parser evaluation.
ProductCache	storeId, queryHash, productCards, expiresAt	Reduces repeated live retrieval.
ClickEvent	queryId, productId, redirectUrl, timestamp	Supports analytics and affiliate tracking.
FailureLog	queryId, domain, module, errorType, message	Supports reliability improvement.

6.6 Known Site Flow and Unknown Site Flow

For a known site, the system can immediately load the stored schema and run the query-to-filter flow. This should be the default production path because it is faster and more reliable. The known-site path is: parse query, load schema, map filters, build URL, retrieve products, normalize, rank, respond.

For an unknown site, the system must first detect whether the store is Shopify or another supported platform. If Shopify markers are found, the discovery worker attempts to identify collections and available filters. The resulting schema is stored with a lower confidence status until

validated. Unknown-site support is important for future scalability, but the academic prototype should not depend entirely on automatic discovery. Manual schema configuration remains a valid fallback.

6.7 Security and Privacy Design

The extension should use minimal host permissions and activate only where needed. The backend should never trust data from the browser. Every request should validate domain, query text, and expected schema identifiers. The system should avoid collecting personal information unless it becomes necessary for future authentication or merchant dashboards.

The scraper should not bypass login walls, payment areas, private APIs, or anti-access controls. For Phase A, the retrieval target is public product discovery pages. This keeps the project aligned with ethical data access and reduces legal risk.

7. Success Metrics and Evaluation Plan

7.1 Evaluation Goal

The evaluation goal is to determine whether the system improves product discovery compared with manual store browsing and whether its technical pipeline behaves reliably enough for a prototype. The system should be evaluated at three levels: parsing and mapping quality, retrieval and ranking quality, and user-facing experience. A successful project does not require perfect universal search; it requires measurable progress against the defined scope [4,7,11,16].

7.2 Quantitative KPIs

Table 13 establishes the quantitative KPIs used to assess prototype success, including target thresholds and measurement methods.

Table 13. Quantitative KPIs for Prototype Evaluation

Metric	Definition	Target for Prototype	Measurement Method
Parser field accuracy	Share of benchmark queries where key fields are extracted correctly.	>= 80% for category and one attribute.	Manual labeled benchmark + unit tests.
Filter matching accuracy	Share of supported attributes mapped to correct site filters.	>= 75% on mapped store.	Expected mapping table comparison.
Query success rate	Share of queries returning at least one relevant product or valid fallback.	>= 70% on benchmark set.	Automated E2E run + manual relevance check.
Top-3 relevance	At least one relevant product appears in first three results.	>= 70% on supported queries.	Human review of benchmark outputs.
Median latency	Time from query submission to response.	<= 5s cached, <= 10s uncached.	Backend timing logs.
Scraping success rate	Share of retrieval attempts that produce normalized product cards.	>= 85% on saved fixtures, >= 70% live.	Integration test results and logs.
Extension stability	No extension crash during normal browsing and repeated queries.	30 minutes continuous use.	Manual and Playwright extension test.
Fallback correctness	Failures return safe messages, not crashes or hallucinated products.	100% for known failure cases.	Regression test suite.
Click-through tracking	Clicked product results generate stored click event.	>= 95% of clicks tracked in demo.	Redirect endpoint integration test.

7.3 Qualitative Evaluation

Quantitative and qualitative evaluation address complementary questions: quantitative metrics establish whether the system performs correctly, while qualitative methods establish whether it is trusted, understandable, and preferred by users - both of which are required to assess a conversational product-discovery system [11,19]. The questionnaires (appendix C) will measure perceived usefulness, ease of use, trust in returned results, clarity of explanations, and willingness to use the assistant again. Open questions will collect examples of confusing outputs, missing features, and cases where manual browsing felt better. [4,11,19]

Qualitative analysis will be used to explain metric failures. For example, a query may be technically successful because it returns products, but users may still dislike the response if it is too verbose, misses an important style constraint, or fails to explain why a product was included [11,19].

7.4 Benchmark Query Set

Table 14 provides a curated benchmark query set covering five query types used to validate parser and retrieval behavior.

Table 14. Benchmark Query Set and Expected Behavior

Query Type	Example	Expected Parser Output	Expected Behavior
Exact attributes	black nike shoes under 400	category=shoes, brand=Nike, color=black, maxPrice=400	Apply supported filters and rank exact matches high.
Use case	laptop for programming student	category=laptop, useCase=programming, audience=student	Use schema and ranking rules to prefer relevant specs.
Ambiguous budget	cheap red hoodie	category=hoodie, color=red, budgetIntent=cheap	Map cheap to low-price sort or max price heuristic.
Unsupported filter	vegan leather boots	category=boots, material=vegan leather	Ignore unsupported filter safely and explain limitation.
No result	green formal shoes size 99	category=shoes, color=green, size=99	Return fallback with no invented products.

8. Testing Process

8.1 Testing Strategy

The testing strategy follows the system architecture. Each module receives isolated tests, then the pipeline is validated through integration and end-to-end tests. The most important principle is that scraping and LLM behavior must be made repeatable. Saved HTML fixtures, mocked LLM responses, and seeded database schemas will be used to create stable tests. Live-site tests will be used as smoke tests rather than the only validation method.

8.2 Unit Tests

Table 15 defines unit test scenarios for each system module, specifying example inputs and the expected isolated behavior.

Table 15. Unit Test Scenarios per Module

Module	Example Test	Expected Result
Query Parser	"black nike shoes under 400"	Extract category, brand, color, maxPrice.
Parser Validation	Empty string or symbols only.	Return validation error or safe fallback object.
Filter Mapper	brand=Nike against store schema.	Map to Nike option with high confidence.
Synonym Logic	trainers against sneakers/shoes schema.	Map to configured synonym when available.
URL Builder	Mapped color + brand filters.	Generate valid Shopify URL or collection path.
Normalizer	Product without image.	Create product card with nullable image, no crash.
Ranker	Exact match vs partial match.	Exact match ranks higher.
Redirect Service	Valid product URL.	Generate tracked redirect and store click event.

8.3 Integration Tests

Integration tests validate interactions between modules. The parser-to-mapper test checks that a ParsedQuery can be mapped against a seeded schema. The mapper-to-url test checks that a valid retrieval URL is produced. The scraper-to-normalizer test checks that saved HTML fixtures produce the expected product card structure. The API-to-database test checks that query logs and click events are stored correctly. The extension-to-backend test checks that a query from the overlay reaches the backend and produces a valid response.

8.4 End-to-End Tests

Table 16 describes the six end-to-end scenarios used to validate the full system pipeline, each with its expected pass criteria.

Table 16. End-to-End Test Scenarios

Scenario	Flow	Pass Criteria
Supported query	Open supported store -> ask query -> receive product cards -> click product.	Relevant products shown and click tracked.
Unsupported filter	Ask for attribute not supported by store schema.	System explains limitation or relaxes filter safely.
Empty result	Ask for impossible product combination.	No hallucinated products; fallback response displayed.
Unknown store	Open store without schema.	Discovery starts or unsupported-site message shown.
Backend failure	Simulate API timeout.	Extension shows safe error state and remains usable.
Cached repeated query	Run same query twice.	Second response uses cache when policy allows.

8.5 Test Tools

Performance and reliability are validated via k6/Artillery latency smoke tests and failure-injection tests (missing schema, parser failure, scraper timeout, LLM unavailability), which must trigger a controlled fallback rather than a crash or invented product.

8.6 Regression Suite

A cross-phase regression suite will be maintained after the first complete flow exists. The suite should include at least twenty benchmark queries, one seeded store schema, several saved HTML fixture pages, expected normalized product cards, expected ranking order for selected cases, and failure fixtures. Each major code change should run this suite before the project demo.

9. Risks, Ethics, and Limitations

9.1 Technical Risks

The first risk is storefront variability. Shopify is a platform, not a single UI. Different themes and apps may render filters differently. The project reduces this risk by starting with manual mapping for one store and by designing automatic discovery as an incremental module [2,5,8,9].

The second risk is retrieval instability. Live websites may change markup, block requests, or load data dynamically. The mitigation is to use saved fixtures for tests, cache recent results, log extraction failures, and keep the scraping layer replaceable by future official APIs or merchant integration [2,5,8,9].

The third risk is LLM unpredictability. LLM outputs may be inconsistent or overly verbose. The mitigation is strict schemas, deterministic fallback rules, prompt constraints, and validation before using model output. The LLM should not directly write product URLs or invent products [4,7,13,21].

9.2 Ethical and Legal Boundaries

The system should retrieve only public product discovery data and should avoid behavior that resembles credential bypass, private data extraction, or excessive automated traffic. The project should respect practical web etiquette: rate limits, caching, clear user-agent identification if appropriate, and immediate fallback when access is blocked [2,5,8,9].

For future commercialization, merchant-approved integration is preferable. A SaaS merchant dashboard, official API access, and agreed analytics would reduce scraping dependency and

improve reliability. In the academic prototype, scraping is a feasibility method, not a justification for unrestricted data extraction [6,11].

9.3 Privacy Risks

The browser extension must avoid collecting unrelated browsing behavior. Query logs are useful for evaluation and analytics, but they should be scoped to product discovery and should not include personal data unless explicitly needed. Session identifiers can be anonymous. If authentication is added later, privacy policy and data deletion processes will be needed.

9.4 Scope Limitations

The Phase A project will not solve universal e-commerce search across all platforms. It will focus on Shopify first, with an architecture that can expand to WooCommerce, Magento, BigCommerce, and PrestaShop later. It will not implement deep user personalization in the first prototype. It will not guarantee real-time inventory accuracy unless the retrieved product page exposes current availability. It will not replace the store checkout or product detail pages [6,11,19].

9.5 Business Risks

Affiliate monetization depends on supported programs, link policies, merchant acceptance, and conversion behavior. The prototype should therefore treat affiliate tracking as a measurement capability first. The most important validation is whether users click returned products and whether merchants could benefit from improved discovery analytics [11,15].

10. Expected Deliverables and Summary

10.1 Expected Deliverables

Table 17 itemizes the Phase A project deliverables, from the prototype application to the final project book.

Table 17. List of Expected Project Deliverables

Deliverable	Description
Prototype application	Chrome extension overlay connected to backend query API.
Backend services	Parser, mapper, URL builder, retriever, normalizer, ranker, response generator, redirect service.
Database	Store schema, filters, cache, query logs, click events, failure logs.
Diagrams	Architecture diagram and activity diagram.
Test suite	Unit, integration, E2E, fixture, and performance smoke tests.
Evaluation report	KPI results, questionnaire summary, limitations, and future work.
Project book	Formal documentation with literature survey, requirements, process, architecture, metrics, and appendices.

10.2 Future Work

- Merchant dashboard for schema configuration and analytics.
- Official Shopify app or merchant-approved integration path.
- Support for WooCommerce, Magento, BigCommerce, and PrestaShop.
- Vector indexing of product descriptions for semantic reranking.
- Extending the current fixed-order reasoning loop toward limited multi-step planning
- Personalized shopping preferences with explicit user consent.
- A/B testing against manual browsing and native store search.
- Improved multilingual support, especially Hebrew and English mixed queries.

10.3 Summary

The system proposes a practical middle path between rigid e-commerce filters and ungrounded generic AI chatbots. Positioned this way, CartPilot is best described as a grounded, agentic retrieval system: it plans and sequences retrieval actions like an agent, but constrains that reasoning loop tightly enough to ensure every returned product originates from retrieved or validated store data. The user receives a conversational interface, while the system preserves structured retrieval and store-specific grounding. The main engineering value is the query-to-filter translation and schema-aware retrieval pipeline. The main user value is faster and clearer product discovery. The main business value is improved exposure of relevant products and measurable engagement.

The project is feasible because it begins with a constrained Shopify-first prototype, uses modular architecture, and defines measurable success criteria. The architecture is further extensible: once a stable single-store deployment is achieved, the same pipeline may be extended to support additional stores, better ranking, merchant dashboards, and SaaS deployment.

Appendix A – AI Tools and Prompt Log

A.1 AI Tools Used

The following table lists the AI tools utilized during the development of this project, specifying each tool's purpose and the type of output it contributed.

Tool	Purpose in Project	Output Used
ChatGPT	Drafting, restructuring, requirement extraction, architecture review, test planning, prompt refinement.	Project book sections, diagrams, tables, and engineering explanations.
Google Scholar / academic search	Finding academic references for product search, RAG, chatbots, and browser extensions.	Literature survey and APA references.
NotebookLM or equivalent document assistant	Optional summarization of long articles and project documents.	Article summaries and comparison notes.
Diagramming assistant / Mermaid	Drafting activity and architecture diagrams.	Diagram scripts and exported figures.
Code assistant	Generating code skeletons, tests, and review checklists.	Implementation support, subject to manual verification.

A.2 Prompt Log

The following table documents the AI prompts used throughout the project, organized by area, to support transparency and reproducibility of AI-assisted development tasks.

Prompt Area	Prompt Text
Problem framing	Analyze the system project idea. Define the core problem, target users, existing alternatives, and why a Chrome extension overlay can solve part of the problem.
Literature search	Find academic sources about e-commerce product search, conversational recommender systems, RAG, LLM query understanding, and browser extensions. Return APA references and key findings.
Requirements extraction	Given the business plan, development plan, and architecture notes, extract functional and non-functional requirements for a software engineering project book.
Architecture drafting	Create a high-level package/deployment architecture for a Shopify-first AI product discovery system using query parsing, filter mapping, scraping, ranking, RAG, and a Chrome extension.
Activity diagram	Create an activity diagram for the flow: user query -> site detection -> schema loading/discovery -> parsing -> filter mapping -> retrieval -> ranking -> response.
Testing plan	Build a testing plan with unit, integration, E2E, fixture-based scraping tests, extension tests, and quantitative KPIs.
Questionnaire refinement	Create concise Likert-scale and open-ended questions for shoppers and business stakeholders to evaluate usefulness, trust, relevance, and usability.
Formal writing	Rewrite the sections in formal academic English while keeping the project practical and engineering-oriented.

Appendix B – References

B.1 Academic References

[1] Balachandran, A., & Masum, M. (2025). VisioRAG: A multimodal framework for enhancing recommendation system using vision transformers and RAG. In 2025 IEEE Conference on Artificial Intelligence (CAI). IEEE. https://ieeexplore.ieee.org/abstract/document/11050521

[2] Chauhan, K. S., & Srivastava, S. (2024). RAG based implementation for smart web scraping using Crawl4ai. Department of Information Technology, Manipal University Jaipur. https://www.researchsquare.com/article/rs-6846502/v1

[3] de Freitas, B. A. T., & Lotufo, R. de A. (2024). Retail-GPT: Leveraging Retrieval Augmented Generation (RAG) for building e-commerce chat assistants. XXXII Congresso de Iniciação Científica, Unicamp. https://arxiv.org/abs/2408.08925

[4] Es, S., James, J., Espinosa-Anke, L., & Schockaert, S. (2024). RAGAS: Automated evaluation of Retrieval Augmented Generation. Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations, 150–158. https://arxiv.org/abs/2309.15217

[5] Gheorghe, M., Mihai, F.-C., & Dârdală, M. (2018). Modern techniques of web scraping for data scientists. Revista Română de Interacțiune Om-Calculator, 11(1), 63–75.

[6] Gupta, A. (2014). E-commerce: Role of e-commerce in today's business. International Journal of Computing and Corporate Research, 4(1).

[7] Imtiyaz, I., Parvaz, M., Mandha, S., Farooq, F., & Kolankeh, A. K. (2025). Assessing RAG: A comprehensive review of evaluation frameworks. In *2025 International Conference on Computational Intelligence and Knowledge Economy (ICCIKE)* (pp. 490–495). IEEE. https://doi.org/10.1109/ICCIKE67021.2025.11318213

[8] Khder, M. A. (2021). Web scraping or web crawling: State of art, techniques, approaches and application. International Journal of Advance Soft Computing and Applications, 13(3). https://doi.org/10.15849/IJASCA.211128.11

[9] Kurniawati, D. (2017). Increased information retrieval capabilities on e-commerce websites using scraping techniques. In 2017 International Conference on Sustainable Information Engineering and Technology (SIET). IEEE. https://ieeexplore.ieee.org/abstract/document/8304139

[10] Liu, Y., Zhang, W.-N., Chen, Y., Zhang, Y., Bai, H., Feng, F., Cui, H., Li, Y., & Che, W. (2023). Conversational recommender system and Large Language Model are made for each other in e-commerce pre-sales dialogue. Findings of the Association for Computational Linguistics: EMNLP 2023, 9587–9605.

[11] Madanchian, M. (2024). The impact of artificial intelligence marketing on e-commerce sales. Systems, 12(10), 429. https://doi.org/10.3390/systems12100429

[12] Mishra, P. P., Yeole, K. P., Keshavamurthy, R., Surana, M. B., & Sarayloo, F. (2025). A systematic framework for enterprise knowledge retrieval: Leveraging LLM-generated metadata to enhance RAG systems. University of Illinois Chicago. arXiv preprint arXiv:2512.05411. https://arxiv.org/abs/2512.05411

[13] Oche, A. J., Folashade, A. G., Ghosal, T., & Biswas, A. (2025). A systematic review of key Retrieval-Augmented Generation (RAG) systems: Progress, gaps, and future directions. arXiv preprint arXiv:2507.18910v1. https://arxiv.org/abs/2507.18910

[14] Rekha, M., Ramachandran, R., Kalyaan Mahendar, R. R., & Bhuvan Chandra, P. (2025). GenCartAI: Next-generation e-commerce using RAG model for intelligent chatbot. In 2025 International Conference on Circuits, Controls and Communications (CCUBE). IEEE. https://ieeexplore.ieee.org/abstract/document/11340290

[15] Rokon, M. O. F., Du, W., Wang, Z., & Wen, M. (2024). Next-gen sponsored search: Crafting the perfect query with inventory-aware RAG (InvAwr-RAG)-based GenAI. In *Proceedings of the 2024 SIGIR Workshop on eCommerce (SIGIR eCom'24)*. CEUR Workshop Proceedings. Walmart AdTech. https://ceur-ws.org/Vol-3843/paper_18.pdf

[16] Sachdev, J., Rosario, S. D., Wen, H., & Kirti, S. (2025). Automated query-product relevance labeling using Large Language Models for e-commerce search. arXiv preprint arXiv:2502.15990v1. https://arxiv.org/abs/2502.15990

[17] Tangarajan, P., Rajasekar, A. A., Rathi, M., Dandin, V. R., & Ersoy, O. (2025). Contextually aware e-commerce product question answering using RAG. arXiv preprint arXiv:2508.01990. https://arxiv.org/abs/2508.01990

[18] Tikhnadhi, K. (2025). Agentic RAG systems for real-time financial decision making: A multi-agent framework with Model Context Protocol integration. In 2025 IEEE 2nd International Conference on Information Technology, Electronics and Intelligent Communication Systems (ICITEICS). IEEE. https://ieeexplore.ieee.org/abstract/document/11341179

[19] Valle, D. X., & Oliveira, H. T. A. (2025). Enhancing e-commerce with a RAG-powered conversational recommender system. Programa de Pós-graduação em Computação Aplicada (PPComp), Instituto Federal do Espírito Santo (IFES). https://sol.sbc.org.br/index.php/eniac/article/view/38713

[20] Xu, C., Kang, Y., Feng, Q., Hua, J., Li, P., Wu, F., Hu, W., Chen, X., Huang, S.-J., & Chen, S. (2026). ItemRAG: Retrieval-augmented generation with item-based knowledge computing for e-commerce product question answering. Big Data Mining and Analytics, 9(2), 407–424. https://ieeexplore.ieee.org/abstract/document/11373501

[21] Xu, T., Jin, Y., Yao, S., Huang, Z., Tang, H., Dong, C., & Ou, D. (2025). DyKnow-RAG: Dynamic knowledge utilization reinforcement framework for noisy Retrieval-Augmented Generation in e-commerce search relevance. arXiv preprint arXiv:2510.11122v1. https://arxiv.org/abs/2510.11122

B.2 Development References

[22] Vercel. (2024). Next.js App Router Documentation. Vercel Inc. https://nextjs.org/docs

[23] Google. (2024). Firebase Firestore Documentation. Google LLC. https://firebase.google.com/docs/firestore

[24] shadcn. (2024). shadcn/ui – Beautifully designed components built with Radix UI and Tailwind CSS. https://ui.shadcn.com

[25] Colinhacks. (2024). Zod: TypeScript-first schema validation with static type inference. https://zod.dev

[26] Microsoft. (2024). TypeScript Documentation. Microsoft Corporation. https://www.typescriptlang.org/docs

Appendix C - Questionnaires

C.1 - Business Questionnaire

Rating Scale

For Parts A and B, please select the number that best reflects your level of agreement:

Value	Meaning
1	Strongly Disagree
2	Disagree
3	Neutral
4	Agree
5	Strongly Agree

Part A | Screening & Background

Basis: Moore & Benbasat (1991); Grandon & Pearson (2004).

Code	Question	Response Options
B1	What best describes your role in the business?	Owner / Store Manager / Digital Marketing Manager / Other
B2	How many years has your e-commerce store been operating?	< 1 yr / 1–3 yrs / 3–7 yrs / > 7 yrs
B3	Which e-commerce platform does your store run on?	Shopify / WooCommerce / Wix / Custom-built / Other
B4	Approximately how many products does your catalog contain?	< 50 / 50–500 / 500–5,000 / > 5,000
B5	How familiar are you with AI tools such as ChatGPT or AI-powered search?	Not at all / Heard of them / Tried once or twice / Regular user

Part B | Evaluation Statements

Please rate your level of agreement with each of the following 15 statements (1 = Strongly Disagree, 5 = Strongly Agree).

Basis: TAM (Davis, 1989); De Cicco et al. (2022); UTAUT2 (Venkatesh et al., 2012); Paraskevi et al. (2023); TRI 2.0 (Parasuraman & Colby, 2015); TXAI (Hoffman et al., 2023); DeLone & McLean (2003); Ulwick (2002); Bilquise et al. (2025).

Code	Topic	Statement	1	2	3	4	5
Q1	Value	A smarter, intent-aware search experience would directly increase sales in my store. [Ulwick, 2002 — Jobs-to-be-Done]					

Code	Topic	Statement	1	2	3	4	5
Q2	Readiness	I believe AI tools can measurably improve the performance of my online store. [TRI 2.0 — Optimism]					
Q3	Readiness	I am comfortable integrating AI-based tools into my business operations. [TRI 2.0 — Optimism]					
Q4	Usefulness	A natural-language search overlay gives customers a better discovery experience than a standard search bar or filter panel. [TAM — Perceived Usefulness]					
Q5	Usefulness	Automatically translating a customer's free-text query into the store's existing filter structure is a capability that would benefit my store. [TAM — Perceived Usefulness]					
Q6	Ease of Use	Installing and enabling a browser-extension-based tool appears straightforward, even without specialist technical knowledge. [TAM — Perceived Ease of Use]					
Q7	Trust	I trust that the system will only return products that genuinely exist in my store's current inventory. [TXAI — Reliability]					
Q8	Trust	Knowing that results are grounded in real-time data retrieved directly from my store's pages — rather than generated from model memory — increases my confidence in the system. [TXAI — Credibility / RAG grounding]					
Q9	Feature	It is important to me that the system explains why a specific product was returned (e.g., "matched your color and price criteria"). [TXAI — Explanation Goodness; Hoffman et al., 2023]					
Q10	Value	I would expect this tool to measurably improve my store's conversion rate or reduce customer drop-off within a few months of deployment. [DeLone & McLean — Net Benefits]					
Q11	Trust	It is important to me that the system explicitly notifies the customer when no matching product is found, rather than generating a fabricated result. [TXAI — Honesty / Anti-hallucination]					
Q12	Pain Point	Customers in my store struggle to find the products they are actually looking for.					
Q13	Pain Point	It is difficult for customers to search using multiple criteria simultaneously (e.g., color + price range + brand in one query).					
Q14	Barrier	I am concerned that the system could present inaccurate or hallucinated product information to my customers. [UTAUT2 / Bilquise — AI Accuracy Risk]					
Q15	Barrier	The ongoing cost of integrating and maintaining this tool is a significant factor in my adoption decision. [UTAUT2 — Price Value]					

Part C | Open-Ended Questions

Please answer briefly in your own words. There are no right or wrong answers.

#	Question	Your Response
OQ1	What is the single biggest barrier preventing you from adopting an AI search tool in your store today?	
OQ2	If this tool worked perfectly, what specific business result would you expect to see within 6 months?	
OQ3	How important is it that this tool supports Shopify specifically, versus also covering WooCommerce or other platforms?	
OQ4	Is there a product search or discovery feature not mentioned in this questionnaire that would be essential for you?	
OQ5	What would need to be true for you to feel confident enough to deploy this tool in your live store?	

Academic References

All questionnaire items are grounded in validated instruments. Topic codes in the evaluation table map directly to entries below.

#	Author(s)	Year	Full Reference + PDF	Maps to
1	**Davis, F.D.**	1989	Perceived usefulness, perceived ease of use, and user acceptance of IT. MIS Quarterly, 13(3), 319–340. Davis1989MISQ.pdf	**Q4, Q5, Q6**
2	**De Cicco, R. et al.**	2022	Understanding Users' Acceptance of Chatbots: Extended TAM. LNCS 13171, Springer. DOI: 10.1007/978-3-030-94890-0_1DeCico.pdf	**Q4, Q5, Q6**
3	**Venkatesh, V., Thong, J.Y., & Xu, X.**	2012	Consumer acceptance and use of IT: Extending UTAUT2. MIS Quarterly, 36(1), 157–178. Venkatesh2012.pdf	**Q2, Q3, Q14, Q15**
4	**Paraskevi, M. et al.**	2023	Modeling Behavioral Intention towards Mobile Chatbot Adoption: UTAUT2 Extension. Human Behavior & Emerging Technologies. DOI: 10.1155/2023/8859989 Paraskevi2023.pdf	**Q14, Q15**

#	Author(s)	Year	Full Reference + PDF	Maps to
5	**Parasuraman, A., & Colby, C.L.**	2015	An Updated and Streamlined Technology Readiness Index: TRI 2.0. Journal of Service Research, 18(1), 59–74. TRI2.0.pdf	**Q2, Q3**
6	**Hoffman, R.R., Mueller, S.T., Klein, G., & Litman, J.**	2023	Measures for explainable AI: trust, satisfaction, and mental models. Frontiers in Computer Science, 5. Hoffman2023.pdf	**Q7, Q8, Q9, Q11 — TXAI scale**
7	**Schaefer, K.E. et al.**	2024	To Trust or Distrust Trust Measures: Validating Questionnaires for Trust in AI. arXiv:2403.00582. (ACM FAccT 2025). Schaefer2024.pdf	**Q7, Q8, Q11 — TXAI validation**
8	**Mayer, R.C., Davis, J.H., & Schoorman, F.D.**	1995	An integrative model of organisational trust. Academy of Management Review, 20(3), 709–734. Mayer1995.pdf	**Q7, Q11 — trust foundations**
9	**DeLone, W.H., & McLean, E.R.**	2003	The DeLone and McLean IS success model: ten-year update. Journal of MIS, 19(4), 9–30. DeLoneMcLean2003.pdf	**Q10**
10	**Grandon, E.E., & Pearson, J.M.**	2004	Electronic commerce adoption: empirical study of US SMEs. Information & Management, 42(1), 197–216. Grandon2004.pdf	**Part A**
11	**Moore, G.C., & Benbasat, I.**	1991	Development of an instrument to measure perceptions of adopting an IT innovation. Information Systems Research, 2(3), 192–222. Moore1991.pdf	**Part A**
12	**Ulwick, A.W.**	2002	Turn customer input into innovation. Harvard Business Review, 80(1), 91–97. Ulwick2002.pdf	**Q1, Q12, Q13**
13	**Rowley, J.**	2000	Product search in e-shopping: a review and research propositions. Journal of Consumer Marketing, 17(1), 20–35. Rowley2000.pdf	**Q12, Q13**
14	**Bilquise, G. et al.**	2025	Driving AI chatbot adoption: A systematic review. ScienceDirect. DOI: 10.1016/j.jjimei.2025.100272 Bilquise2025.pdf	**Q1, Q14**
15	**Parasuraman, R., & Riley, V.**	1997	Humans and automation: Use, misuse, disuse, abuse. Human Factors, 39(2), 230–253. Parasuraman1997.pdf	**Q7 — autonomy/control background**
16	**Sheehan, B., Jin, H.S., & Gottlieb, U.**	2020	Customer service chatbots: Anthropomorphism and adoption. Journal of Business Research, 115, 14–24. Sheehan2020.pdf	**Q4, Q6**
17	**Cheng, Y., & Jiang, H.**	2020	How do AI-driven chatbots impact user experience? Journal of Broadcasting & Electronic Media, 64(4), 592–614. Cheng2020.pdf	**Q4, Q5**

Note: All Likert-scale items have been adapted from validated instruments and contextualised for a RAG-based, natural-language product search system delivered via a browser extension overlay on Shopify and other e-commerce platforms. The product name has been intentionally omitted from all questionnaire items to prevent respondent bias

C.2 - User Survey

Rating Scale

Please select the number that best reflects how much you agree with each statement:

Value	Meaning
1	Strongly Disagree — I completely disagree with this statement
2	Disagree — I tend to disagree
3	Neutral — I am unsure or it does not really apply to me
4	Agree — I tend to agree
5	Strongly Agree — I completely agree with this statement

Part A | About You

Basis: Knijnenburg et al. (2012) — personal characteristics as moderators of system experience.

Code	Question	Response Options
S1	What is your age group?	Under 18 / 18–24 / 25–34 / 35–49 / 50+
S2	How often do you shop online?	Rarely (less than once a month) / Sometimes (1–3 times/month) / Often (weekly or more)
S3	How do you usually search for products in an online store?	I browse categories / I use the search bar / I use filters / A mix of all of these
S4	Have you ever used an AI assistant (e.g., ChatGPT, Alexa, Google Assistant) to help you find products?	Never / Once or twice / A few times / I use it regularly
S5	How comfortable are you with installing browser tools or extensions?	Not comfortable at all / A little uncomfortable / Neutral / Comfortable / Very comfortable

Part B | Your Needs & Expectations

The following statements are about your current online shopping experience and your expectations of a tool like the system. Please rate each one from 1 (Strongly Disagree) to 5 (Strongly Agree).

Basis: Pain Points — Rowley (2000), Ulwick (2002); Expected Usefulness — Davis (1989), De Cicco et al. (2022); Technology Readiness — Parasuraman & Colby (2015); Pre-use Trust — Mayer et al. (1995), Hoffman et al. (2023); Adoption Intention — Venkatesh et al. (2012).

Code	Dimension	Statement	1	2	3	4	5
Q1	Pain Point	I often struggle to find exactly what I am looking for when searching in an online store. [Rowley, 2000 — Search difficulty]					
Q2	Pain Point	I find it frustrating to use multiple filters or browse many categories just to describe one simple product I want. [Ulwick, 2002 — Jobs-to-be-Done: unmet need]					
Q3	Pain Point	I have left an online store without buying because I could not find the product I was looking for, even though I believed it was there. [Rowley, 2000 — Search abandonment]					
Q4	ExpectedUsefulness	I would find it useful to describe what I am looking for in my own words (e.g., "warm blue jacket under $60") rather than using search bars and filters. [TAM — Perceived Usefulness as expectation]					
Q5	ExpectedUsefulness	I expect a natural-language search tool to find more relevant products than a standard keyword search bar. [TAM — Performance expectancy]					
Q6	ExpectedUsefulness	Being able to describe a product in plain language would save me time and effort when shopping online. [TAM — Perceived Usefulness; De Cicco et al., 2022]					
Q7	TechReadiness	I am open to trying an AI-powered browser tool that helps me find products using natural language. [TRI 2.0 — Optimism]					
Q8	TechReadiness	I am willing to try new digital tools to improve my online shopping experience, even if I have not used something similar before. [TRI 2.0 — Innovativeness]					
Q9	Pre-useTrust	I would trust an AI search tool more if I knew it only shows me products that actually exist and are available in the store I am browsing. [Mayer et al., 1995 — Ability-based trust]					
Q10	Pre-useTrust	I would want the tool to explain why specific products appeared for my search — not just show results without any reason. [Hoffman et al., 2023 — TXAI: Explainability as trust driver]					
Q11	Pre-useTrust	I am concerned that an AI search tool might show me products that do not actually match what I asked for. [Mayer et al., 1995 — Perceived risk / benevolence]					
Q12	Pre-useTrust	I would feel more confident using an AI search tool if I knew its results come directly from the store's real product pages, not from the AI's own memory. [Hoffman et al., 2023 — Credibility; RAG grounding]					
Q13	AdoptionIntention	If the system were available on an online store I visit, I would be willing to try it. [UTAUT2 — Behavioural Intention]					
Q14	AdoptionIntention	I would use a browser tool that helps me search for products using plain language, as long as it is					

Code	Dimension	Statement	1	2	3	4	5
		easy to install and use. [UTAUT2 — Effort Expectancy]					
Q15	AdoptionIntention	A tool that automatically converts my description into the store's search filters would make my online shopping experience meaningfully better. [UTAUT2 — Performance Expectancy]					

Part C | Your Own Words

Please answer each question briefly. A sentence or two is enough — we just want your honest reaction.

#	Question	Your Answer
OQ1	When you search for a product in an online store, what is the most frustrating part of the experience?	
OQ2	Describe a time when you could not find a product online even though you believed the store had it. What happened?	
OQ3	If you could search for any product by just describing it in your own words, how would you describe your ideal search experience?	
OQ4	What would you need to know about an AI search tool before you felt comfortable trusting its results?	
OQ5	What concerns or reservations would you have about using a browser tool like the system?	

Academic References

This is a pre-use needs-assessment instrument. Unlike CRS-Que (which requires prior system interaction), the constructs below are validated for measuring user needs, expectations, readiness, and pre-adoption trust — all without prior exposure to the evaluated system.

#	Author(s)	Year	Full Reference	Maps to
1	**Rowley, J.**	2000	Product search in e-shopping: a review and research propositions. Journal of Consumer Marketing, 17(1), 20–35.	**Q1, Q3(Search difficulty & abandonment)**
2	**Ulwick, A.W.**	2002	Turn customer input into innovation. Harvard Business Review, 80(1), 91–97.	**Q2(Jobs-to-be-Done — unmet need)**
3	**Davis, F.D.**	1989	Perceived usefulness, perceived ease of use, and user acceptance of information technology. MIS Quarterly, 13(3), 319–340.	**Q4, Q5, Q6(Expected Usefulness — TAM)**
4	**De Cicco, R. et al.**	2022	Understanding Users' Acceptance of Chatbots: An Extended TAM Approach. LNCS 13171, Springer. DOI: 10.1007/978-3-030-94890-0_1	**Q4, Q6(TAM for conversational tools)**
5	**Parasuraman, A., & Colby, C.L.**	2015	An Updated and Streamlined Technology Readiness Index: TRI 2.0. Journal of Service Research, 18(1), 59–74.	**Q7, Q8(Optimism & Innovativeness)**
6	**Mayer, R.C., Davis, J.H., & Schoorman, F.D.**	1995	An integrative model of organisational trust. Academy of Management Review, 20(3), 709–734.	**Q9, Q11(Ability & Benevolence trust)**
7	**Hoffman, R.R., Mueller, S.T., Klein, G., & Litman, J.**	2023	Measures for explainable AI: trust, satisfaction, and mental models. Frontiers in Computer Science, 5.	**Q10, Q12(Explainability & Credibility as pre-trust)**
8	**Venkatesh, V., Thong, J.Y., & Xu, X.**	2012	Consumer acceptance and use of IT: Extending UTAUT2. MIS Quarterly, 36(1), 157–178.	**Q13, Q14, Q15(Behavioural Intention, Effort & Performance Expectancy)**
9	**Knijnenburg, B.P. et al.**	2012	Explaining the User Experience of Recommender Systems. User Modeling and User-Adapted Interaction, 22(4), 441–504.	**Part A (S1–S5)(Personal characteristics)**

Note: This questionnaire is a pre-use needs-assessment instrument. Respondents have not interacted with the system prior to completing it. Constructs measuring system performance (e.g., CRS-Que accuracy, understanding, satisfaction) are therefore not applicable here and are reserved for the separate post-use evaluation questionnaire administered after the first system session.

Appendix D - Interview Summaries

This appendix summarizes the five stakeholder interviews referenced in Section 3.1 and Section 4.4. Full transcripts and derived requirement lists are available as supplementary material.

Interview 1 – Prospective computer-hardware retailer (age 50, industry veteran, technical). Two decades of hardware retail experience, including managing KSP's operations; planning a professional-grade hardware store. Identified "information overload" as the core problem: customers describe a need ("quiet video-editing PC under 7000") rather than technical specs, and existing search only works once the customer already knows a model. Key requirements: natural-language-to-spec translation grounded in real stock, fast response times (explicitly rejecting multi-step "deep search" delays), per-store data isolation for a future SaaS model, resilient scraping (retries/proxies/user-agents), price/availability history tracking, and the ability to bias recommendations toward higher-margin products under equivalent specs.

Interview 2 – Streetwear & exclusive fashion store owner (age 28, non-technical, Shopify). Sells limited-edition streetwear through a website and Instagram. Customers search by style and vibe ("vintage oversized faded black hoodie") rather than category, and manually answering repeat questions (stock, restock timing, similar items) is time-consuming. Key requirements: style-aware search rather than keyword matching, strict grounding in current stock (no recommending sold-out or non-existent items), fast mobile response, page-context awareness (e.g., "something similar in white" while viewing a specific product), search analytics, clear fallback messaging for unsupported sites, and a Chrome-extension deployment to avoid touching site code. Consistency of answers was raised as more important than adding AI for its own sake.

Interview 3 – Multi-store biomedical equipment owner (technical background, Shopify). Operates four Shopify stores selling biomedical and lab equipment to distinct clinical/lab/B2B segments, each with overlapping but non-identical catalogs, SKUs, and regulatory considerations. Customers often search by use-case or informal terms rather than the store's medical/manufacturer categorization. Key requirements: schema-aware query mapping (collections, tags, metafields, vendors), strict real-time stock accuracy, a hybrid model (cached schema + live query-time verification rather than full pre-scraping), full multi-store data isolation, transparency into which filters/confidence produced a result, manual override of query-to-filter mappings, analytics on failed searches and conversions, and explicit guardrails against clinical/medical claims — the system should present catalog matches only, never medical advice.

Interview 4 – Footwear & fashion store owner (light-technical). Sells sneakers and everyday fashion mainly through Instagram and word-of-mouth, and manually answers repeated stock/size/style questions. Key requirements: accurate, inventory-grounded answers (no promising sizes/items that aren't in stock), simple non-technical deployment ("something that just works" or fully managed), Chrome-extension delivery, similar-item recommendations for out-of-stock sizes, simple analytics (top searches, unmet demand), mobile-first performance, and clear "not found" fallback messaging rather than invented answers.

Interview 5 – Candles & essential oils store owner (age 23, non-technical, Shopify). Runs a small handmade-goods store (~60 base products with many size/scent variants) driven largely by Instagram traffic. Customers frequently describe needs in natural language ("something calming for nighttime") that do not map to existing filters (scent, size), forcing manual handling of repeat questions. Key requirements: natural-language-to-filter mapping, real-time inventory grounding to avoid hallucinated recommendations, mobile-first performance, simple no-setup deployment, and basic analytics on failed searches.